

Sprint 3 — Data Systems and Data Modelling

Answer Key & Cheat Sheet — correct answers only, for quick review

- 1** A client services dashboard is meant to show every client alongside any current account details, while still keeping clients with no account row visible and leaving the account fields as NULLs. The report has started returning only matched clients after a recent query rewrite. Which join pattern is required to keep the intended behaviour?

Select ONE

CORRECT ANSWER

- ✓ LEFT JOIN from clients to accounts, so unmatched clients remain in the result and the account columns are NULL where no match exists

- 2** A portfolio review query originally used a LEFT JOIN from clients to accounts so that a client with no account still appeared in the output. A developer changes the join to INNER JOIN and now a subset of clients disappears. Which statements explain the behaviour accurately? (Select all that apply.)

Select ALL that apply

CORRECT ANSWERS

- ✓ A client with no matching row in accounts has nothing to match against under an INNER JOIN, so that client is excluded entirely
- ✓ INNER JOIN only returns rows where a match exists in both tables
- ✓ Switching back to a LEFT JOIN from clients to accounts would restore the client rows, with NULLs in the account columns for unmatched records

- 3** An advisor coverage report must show only advisors who manage three or more clients. The team wants the threshold to be applied after grouping, not before. Which SQL pattern is the correct implementation of that rule?

Select ONE

CORRECT ANSWER

- ✓ GROUP BY advisor_id combined with HAVING COUNT(*) >= 3

- 4** A trading operations team needs a report that shows each account and the date of its latest transaction, regardless of whether the account has just one event or many. A developer proposes a correlated subquery to compare each account row against transaction rows. Which statements are accurate about that approach relative to a scalar subquery used across all rows? (Select all that apply.)

Select ALL that apply

CORRECT ANSWERS

- ✓ This is why correlated subqueries are generally considered more expensive than simple scalar subqueries, though a real query planner may optimise the actual execution
- ✓ A scalar subquery computing a single overall value (with no reference to the outer row) can be evaluated once and reused across every row
- ✓ A correlated subquery references a column from the outer query, so conceptually it needs to be evaluated once per outer row

- 5** A BI analyst builds a query with a CTE to compute each client's total BUY value, then filters to clients above the average total. A second engineer rewrites the same logic as a derived table subquery in the FROM clause, and both versions return the same numbers. In a production codebase, what is the most defensible reason to choose the CTE version?

Select ONE

CORRECT ANSWER

- ✓ They are functionally equivalent for this kind of query, but the named CTE is often easier to read and maintain as the query grows, whereas deeply nested derived tables become harder to reason about and debug

- 6** A colleague proposes replacing a carefully written CTE-based query with a window-function version because it looks more modern. The team wants to evaluate it responsibly before adopting it. Which responses reflect good practice? (Select all that apply.)

Select ALL that apply

CORRECT ANSWERS

- ✓ Compare it against the original for correctness first, confirming that the result set still matches the required business logic
- ✓ Judge readability and maintainability in the context of the team and the specific query, rather than assuming a different formulation is automatically better
- ✓ Only accept it if it is genuinely clearer, faster, or otherwise materially better — not just because it is a different approach

- 7** A data migration team is normalising an export from a legacy reporting feed where each row repeats the advisor's name and the model portfolio's target weight for a given instrument. The team asks why this repeated data matters if the report still looks correct. What is the most specific risk in the current structure?

Select ONE

CORRECT ANSWER

- ✓ If a fact such as an advisor's name or a target weight changes in one row but not the others, the same real-world fact becomes contradictory across the dataset, making catches and reporting unreliable

- 8** While normalising a flat portfolio file, you find a field like `instrument_sector_name` that depends on the instrument rather than on the full composite key of (`portfolio_id`, `instrument_id`). Which statements correctly describe the problem and the fix? (Select all that apply.)

Select ALL that apply

CORRECT ANSWERS

- ✓ This is a partial dependency, because the value depends on only part of the composite key
- ✓ Resolving it takes the schema to 2NF
- ✓ Moving the attribute into a separate table keyed on the instrument resolves the dependency

- 9** A second pass over the model portfolio schema reveals that `advisor_region` depends on `advisor_id` rather than on the client's own primary key. Which statements accurately describe the issue and the correct normalisation step? (Select all that apply.)

Select ALL that apply

CORRECT ANSWERS

- ✓ This is a transitive dependency, because the region depends on `advisor_id`, which is itself a non-key attribute
- ✓ Resolving it brings the schema to 3NF

- 10** You are designing a `client_holdings` table with `client_id`, `instrument_id`, `quantity`, and `as_of_date` to represent the state of a client's portfolio at a given point in time. A colleague argues that a surrogate id is always better than a composite natural key. What is the strongest design argument for choosing (`client_id`, `instrument_id`, `as_of_date`) as the primary key here?

Select ONE

CORRECT ANSWER

- ✓ The natural key reflects the true business identity of the record: a unique client/instrument/date combination prevents duplicate rows for the same holding snapshot and keeps the primary key aligned with the meaning of the data

- 1**
1 A portfolio platform uses an instrument master table with `instrument_id` as the database key and ticker as a business identifier. A colleague proposes adding a separate **UNIQUE** constraint on ticker even though the table already has a primary key. Which statements explain why both constraints are useful in practice? (Select all that apply.)

Select ALL that apply

CORRECT ANSWERS

- ✓ The primary key provides a stable internal identifier that other tables can use in foreign keys and joins
- ✓ The **UNIQUE** constraint on ticker enforces the business rule that two instruments cannot share the same ticker symbol

- 1**
2 A data team adds a **CHECK** constraint so `model_portfolio_holdings.target_weight_pct` must stay within 0 and 100 inclusive, then tries to import a row with a value of 150 during a portfolio refresh. What should happen?

Select ONE

CORRECT ANSWER

- ✓ The insert is rejected and Postgres returns an error indicating the constraint has been violated, because 150 is outside the valid range

- 1**
3 An operations team is reviewing a schema in which every foreign key column has an index except one, on an audit table that is written heavily and almost never queried by join or filter conditions. A developer proposes adding the index anyway to keep the design consistent. What's the best practical objection?

Select ONE

CORRECT ANSWER

- ✓ Every extra index adds write overhead because the index also has to be maintained on inserts and updates, so a rarely used index can become a needless cost without delivering a matching read benefit

- 1**
4 A compliance stakeholder sends a one-line request: "Can we get a way to see which clients have drifted significantly from their model portfolio? We keep finding out too late." No one is available for clarifying questions today. Which of the following shows a sound approach before writing schema changes or SQL? (Select all that apply.)

Select ALL that apply

CORRECT ANSWERS

- ✓ Document the assumptions you are making about what 'drifted significantly' means, and note the missing clarifications instead of guessing silently
- ✓ Write down the questions you would ask if the stakeholder were available, so the team can validate the requirement before building the wrong thing
- ✓ Only after that, identify which existing tables and columns could answer the question and whether a new model is actually needed

- 15** Before building the "portfolio drift" report, the team drafts a statement in plain English: "If a client is more than X percentage points over target in a single fund, that would show up on this report — is that the right interpretation?" What purpose does this step serve?

Select ONE

CORRECT ANSWER

- ✓ It checks whether the team's interpretation of an ambiguous requirement matches the stakeholder's actual intent before they spend time building the wrong solution

- 16** A login service stores tokens for tens of thousands of active sessions, with a requirement to answer a simple question in milliseconds: "does this token still exist and remain valid?" Sessions expire automatically after inactivity. Which statements are genuine reasons a key-value store fits this access pattern well? (Select all that apply.)

Select ALL that apply

CORRECT ANSWERS

- ✓ Key-value stores are designed for very high-frequency, simple key-based reads and writes at concurrency levels typical of session caches
- ✓ Built-in TTL or expiry support matches the requirement that sessions be invalidated automatically after inactivity
- ✓ The access pattern is a single-key lookup with no need for joins, aggregation, or filtering on multiple attributes

- 17** A market-data pipeline ingests millions of price ticks a second across thousands of instruments, and the dominant access pattern is "give me the price history for this instrument over a time range" rather than "show me one full tick and all of its related metadata." Which statements correctly explain why a columnar store is a better fit than a row-oriented store for this workload? (Select all that apply.)

Select ALL that apply

CORRECT ANSWERS

- ✓ A time-range query over one instrument reads a narrow set of relevant columns, which matches column-oriented storage well
- ✓ Columnar formats compress particularly well when columns contain long runs of similar values, which reduces the amount of data read for analytical queries

- 18** A client-facing financial goals product allows users to create goals such as a house deposit, retirement, or education funding, but each goal type has a different set of fields and the product team adds new goal types over time. Which storage model fits this requirement most naturally, and why?

Select ONE

CORRECT ANSWER

- ✓ A document store, because each goal can have a different structure and the schema can evolve without repeated, disruptive migrations for every new type

19 You are onboarding to a cloud data warehouse after working with psql and pgAdmin on a live transactional Postgres database. Which statements accurately describe a genuine trade-off of moving reporting workloads into a separate warehouse? (Select all that apply.)

Select ALL that apply

CORRECT ANSWERS

- ✓ Warehouse data is often refreshed on a schedule rather than reflecting every transaction immediately, so it trades some freshness for analytical scale
- ✓ Reports may lag behind the live system, which is a meaningful drawback when the use case genuinely needs near-real-time answers

20 A colleague explains that a cloud data warehouse separates storage and compute, unlike a single-server Postgres instance where both are tightly coupled. What is the practical advantage they are describing?

Select ONE

CORRECT ANSWER

- ✓ Storage and compute can be scaled independently: a larger virtual warehouse can handle a heavy analytical job without resizing the underlying data store, and compute can be paused when it is not needed